

41052 - Advanced Algorithms  
Programming Assignment 1 - Fast Fourier Transform

Tung Nguyen - 24666048

**Contents**

|          |                                              |          |
|----------|----------------------------------------------|----------|
| <b>1</b> | <b>A little history</b>                      | <b>2</b> |
| <b>2</b> | <b>Relevant background knowledge</b>         | <b>2</b> |
| <b>3</b> | <b>Fast Fourier Transform Implementation</b> | <b>3</b> |
| <b>4</b> | <b>The Empirical Study</b>                   | <b>3</b> |
| <b>5</b> | <b>Lesson learned</b>                        | <b>3</b> |
| <b>6</b> | <b>AI usage Declaration</b>                  | <b>3</b> |

# 1 A little history

In 1802, Napoleon Bonaparte appointed Joseph Fourier as the Prefect of Isère in Grenoble, France. Managing regional public works by day, Fourier spent his nights studying heat diffusion. Starting from Newton's Law of Cooling, he showed how arbitrary functions could be broken down into trigonometric series and integral transform pairs. He presented his findings to the Institut de France in 1807, but the review committee, including Joseph-Louis Lagrange and Pierre-Simon Laplace, held it back due to doubts over mathematical rigour. Only in 1822, after becoming Permanent Secretary of the Académie des Sciences, did Fourier formally publish his treatise *Théorie analytique de la chaleur*, establishing the continuous Fourier Transform.

The discrete version actually predated Fourier's work. In 1759, Alexis Clairaut published an early discrete trigonometric formula while calculating celestial orbits. In 1805, Carl Friedrich Gauss refined this technique to track the paths of asteroids, effectively devising both the discrete Fourier formula and an early Fast Fourier Transform (FFT). Because Gauss recorded his findings in unpublished Latin notes, his work remained largely overlooked until long after his death.

The modern breakthrough occurred during the Cold War. As the United States and the Soviet Union negotiated nuclear test bans, verifying compliance required distinguishing clandestine underground nuclear tests from natural earthquakes. Processing the flood of discrete seismic sensor data was computationally impractical using standard matrix methods, which scaled at  $\mathcal{O}(N^2)$ .

Physicist and defence adviser Richard Garwin recognised this bottleneck and raised it with statistician John Tukey from Princeton and Bell Labs. Tukey proposed a divide-and-conquer approach to accelerate the computation and worked with IBM mathematician James Cooley, who implemented and programmed the algorithm. Published in 1965, the Cooley–Tukey FFT slashed the computational cost to  $\mathcal{O}(N \log N)$  (fundamentally, Gauss invented the same algorithm but his work went unnoticed). By that time, major countries had already created their nuclear devices and went ahead with their own testing.

Nowadays, the FFT has revolutionised digital signal processing, telecommunications and numerical computing worldwide and become one of the most important algorithms in our lives, as it is used every phone and personal device and so much more.

# 2 Relevant background knowledge

The Discrete Fourier Transform (DFT) is a function that receives an array input of complex numbers and returns an array with the same amount of elements. The DFT can be defined as an algorithm.

---

**Algorithm 1** Discrete Fourier Transform

---

**Input:** Input sequence  $x = [x[0], x[1], \dots, x[N-1]]$  of length  $N$

**Output:** Transformed frequency array  $X = [X[0], X[1], \dots, X[N-1]]$

1: Allocate array  $X$  of size  $N$ , initialised to 0

2: **for**  $k \leftarrow 0$  **to**  $N-1$  **do**

3:     **for**  $n \leftarrow 0$  **to**  $N-1$  **do**

4:          $\theta \leftarrow \frac{2\pi}{N} \cdot k \cdot n$

5:          $W \leftarrow \cos(\theta) - i \sin(\theta)$

6:          $X[k] \leftarrow X[k] + x[n] \cdot W$

7:     **end for**

8: **end for**

9: **return**  $X$ 

---

Looking deeper into this algorithm, the first for loop (line 2) is grabbing every element of the input array of size  $N$  (line 6, with  $X[k]$ ). The second for loop (line 3) is also looping over size  $N$  to compute for each value of array  $X$ . Thus, the DFT has a time complexity of  $\mathcal{O}(N^2)$ .

The Inverse Discrete Fourier Transform (IDFT) is used for reversing a transformed array to its original array, essentially undoing the DFT. It can also be described as an algorithm.

---

**Algorithm 2** Inverse Discrete Fourier Transform

---

**Input:** Transformed frequency array  $X = [X[0], X[1], \dots, X[N-1]]$  of size  $N$

**Output:** Original sequence  $x = [x[0], x[1], \dots, x[N-1]]$

```
1: Allocate array  $x$  of size  $N$ , initialised to 0
2: for  $n = 0$  to  $N - 1$  do
3:   for  $k = 0$  to  $N - 1$  do
4:      $\theta = \frac{2\pi}{N} \cdot k \cdot n$ 
5:      $W = \cos(\theta) + i \sin(\theta)$ 
6:      $x[n] = x[n] + X[k] \cdot W$ 
7:   end for
8:    $x[n] = \frac{1}{N} \cdot x[n]$ 
9: end for
10: return  $x$ 
```

---

Once again, due to the nested for loops (line 2 and 3), the IDFT has a complexity of  $\mathcal{O}(N^2)$ .

A Fast Fourier Transform (FFT) is a class of algorithms that can compute the DFT and iDFT faster than  $\mathcal{O}(N^2)$ . The most famous FFT algorithm is the Cooley-Tukey FFT (also known as the Radix-2 FFT) mentioned above.

---

**Algorithm 3** Cooley-Tukey Radix-2 FFT

---

**Input:** Input sequence  $x = [x[0], x[1], \dots, x[N-1]]$  of length  $N = 2^m$

**Output:** Transformed frequency array  $X = [X[0], X[1], \dots, X[N-1]]$

```
1: if  $N = 1$  then
2:   return  $x$ 
3: end if
4: Allocate array  $x_{\text{even}}$  of length  $\frac{N}{2}$ 
5: Allocate array  $x_{\text{odd}}$  of length  $\frac{N}{2}$ 
6: for  $k = 0$  to  $\frac{N}{2} - 1$  do
7:    $x_{\text{even}}[k] \leftarrow x[2k]$ 
8:    $x_{\text{odd}}[k] \leftarrow x[2k + 1]$ 
9: end for
10:  $E \leftarrow \text{FFT}(x_{\text{even}})$  # Recursive call on even-indexed elements
11:  $O \leftarrow \text{FFT}(x_{\text{odd}})$  # Recursive call on odd-indexed elements
12: Allocate array  $X$  of length  $N$ 
13: for  $k \leftarrow 0$  to  $N/2 - 1$  do
14:    $W \leftarrow \exp(-i \frac{2\pi}{N} \cdot k)$ 
15:    $t \leftarrow W \cdot O[k]$ 
16:    $X[k] \leftarrow E[k] + t$ 
17:    $X[k + N/2] \leftarrow E[k] - t$ 
18: end for
19: return  $X$ 
```

---

### 3 Fast Fourier Transform Implementation

### 4 The Empirical Study

### 5 Lesson learned

### 6 AI usage Declaration